DDMR: Dual Dependency-Aware Multilscale Representation Learning for Time Series Forecasting

sole author

Abstract

In time series forecasting (TSF), current mainstream models are primarily MLP- or Transformer-based. MLP-based models are effective and efficient in capturing temporal dependencies, while Transformer-based models, despite higher computational costs, excel at modeling complex interchannel dependencies. Time series often exhibit both high-frequency fluctuations and low-frequency trends, whose superposition increases the difficulty of modeling. Additionally, distribution shifts make accurate forecasting difficult. To address these challenges, we propose the Dual Dependency-Aware Multiscale Representation Learning (DDMR) framework. DDMR uses average pooling to decompose the input into multiple scales. To balance efficiency and expressive power, a dual-stream structure is introduced: an MLP-based linear stream captures temporal dependencies, while a Transformer-based nonlinear stream models channel dependencies. The linear stream incorporates stacked Temporal Dependency Model (TDM) blocks for global-local dependency modeling, to address non-stationarity. In the non-linear stream, Fast Fourier Transform (FFT) projects temporal data into frequency domain, and gated units compute sparse attention to enhance efficiency. Furthermore, we design a Multi-Scale Aggregator (MA) block to integrate multiscale features. To mitigate the effects of distribution shifts during prediction, a Lightweight Mixture-of-Experts (LMoE) block is introduced, combining shared low-level parameters and expert-specific independence for function diversity. Extensive experiments demonstrate that DDMR achieves superior performance on multiple benchmarks.

1 Introduction

Multivariate time series forecasting plays a crucial role in numerous practical applications, including financial investment (Nelson, Pereira, and De Oliveira 2017), energy management (Kong et al. 2017), weather prediction (Rasp et al. 2020), and traffic optimization (Ma et al. 2015), among others.

The emergence of Transformer has brought about a revolution in the field of time series prediction(Vaswani et al. 2017). With its self-attention mechanism, the Transformer model demonstrates strong capabilities in modeling long-range dependencies and complex dynamics, and has become one of the current mainstream tools for time series modeling(Zhou et al. 2021a; Wu et al. 2021; Zhou et al. 2022). However, the Transformer structure typically contains a large number of parameters and has high computational complexity, sometimes making it difficult to meet the requirements of real-time and resource-constrained environments. In contrast, the MLP has regained the attention of researchers due to its simple structure and high computational efficiency. In recent years, with the update of modeling concepts, more and more studies have found that by designing appropriate input representations and modeling structures, models based on MLP cannot only achieve efficient prediction but have even outperformed Transformer on multiple public benchmark datasets, demonstrating great potential. This trend prompts researchers to re-examine the complexity of time series modeling and seek an effective balance between model lightweighting and modeling capabilities.

However, most current methods based on MLP mainly focus on modeling in the time dimension(Hu et al. 2025; Wang et al. 2024), especially for modeling the trends and seasonality in time series, but they lack effective modeling mechanisms for the complex dependencies among channels in multivariate time series. The channel dependence is usually manifested as nonlinear and dynamic interaction relationships between variables, with strong context sensitivity and structural heterogeneity. This makes simple fully connected structures difficult to fully extract the potential correlations between channels. Therefore, although MLP has advantages in computational efficiency, it still has significant deficiencies in capturing the deep relationships between channels, which limits its wide application in multivariate time series modeling.

Furthermore, time series often simultaneously contain short-term high-frequency fluctuations and long-term low-frequency trends. The high-frequency components may interfere with the stable modeling of the long-term trend by the model, while the low-frequency components may mask the key short-term dynamics. The superposition of these multi-frequency information significantly increases the complexity of modeling and the uncertainty of prediction. Therefore, in the process of model design, it becomes particularly crucial to simultaneously consider the high-frequency and low-frequency signals in the time series. In recent years, multi-frequency (or multi-scale) modeling methods have demonstrated importance in the LTSF field(Hu et al. 2025; Wang et al. 2024), effectively enhancing the model's ability to per-

ceive information of different time granularities. However, the existing literature mainly applies multi-frequency modeling to the construction of time-dependent relationships, and there is still a lack of in-depth exploration of the modeling of inter-channel dependencies.

Based on the above observations, this paper, while taking into account both the modeling capability of the model and lightweight design, proposes a dual-stream structure framework to model the variable dependencies and time dependencies in multivariate time series separately. Specifically, we first decompose the original input sequence into multiple frequency components to capture its multi-scale dynamic characteristics. In the nonlinear modeling stream, we adopt a Transformer-based structure to depict the complex dependencies between variables. Considering that time series often exhibit non-stationarity, we introduce the Fast Fourier Transform (FFT) to convert the sequence from the time domain to the frequency domain, thereby enhancing the model's ability to model stable channel dependencies. In the linear modeling stream, we decompose the multi-frequency sequence based on Exponential Moving Average (EMA), extracting the seasonal components and trend components. For the seasonal components, we employ the stacked Temporal Dependency Modeling (TDM) blocks, which progressively decomposes the time series into subsequence-level patches of varying sizes, leveraging the concept of channel-wise independence, to achieve global dependency modeling in the time dimension, while enhancing the expression ability of local patterns; For the trend components, we combine MLP with average pooling operations to further strengthen their trend features. After modeling the channel- or time-dependence in multi-scale sequences, we introduce a Multi-Scale Aggregator (MA) block—an MLP-based structure designed to capture high-order interactions across different scales. Finally, to address distribution shifts, we design a Lightweight Mixture-of-Experts (LMoE) block, where experts share a common low-level backbone while each retains the ability to model time series from different distributions.

We summarized our main contributions as follows:

- We propose a dual-stream architecture that leverages a Transformer-based stream to model channel dependencies in multivariate time series, and an MLP-based stream to capture temporal dependencies. This design strikes an effective balance between modeling capacity and computational efficiency.
- In the Transformer-based stream, we use FFT to project the time-domain information into the frequency domain to address non-stationarity, and model the channel dependency based on the frequency-domain information. To enhance the generalization ability of the model and reduce the computational cost, we introduced gated units into the self-attention mechanism to construct a sparse attention matrix.
- In the MLP-based stream, we overcame the limitation of a single patch size by adopting stacked Temporal Dependency Modeling (TDM) blocks. Each block sequentially uses patches of different sizes to capture global dependencies while enhancing local details.
- We analyzed the impact of different frequency components on prediction and utilized the Multi-Scale Aggregator (MA) block to adaptively aggregate multi-scale information. Finally, we employed the Lightweight Mixture-of-Experts (LMoE) block to model the diverse distributions present in time series data.

2 Related Works

2.1 Time Series Forecasting

In recent years, TSF methods based on deep learning have received extensive attention. Among these, common models include CNN(Lea et al. 2016; Stitsyuk and Choi 2025), RNN(Smyl, Dudek, and Pełka 2021; Hou and Yu 2024), GNN(Wu et al. 2020; Xu, Kosma, and Vazirgiannis 2023), Transformers(Zhou et al. 2021a; Wu et al. 2021; Zhou et al. 2022), and MLP(Hu et al. 2025; Wang et al. 2024; Zeng et al. 2022).

In the Transformer-based models, Informer(Zhou et al. 2021a) addresses the high time and space complexity issues of the original Transformer in long sequence prediction tasks by proposing the ProbSparse self-attention mechanism, which can significantly reduce computational costs while maintaining the prediction accuracy. Additionally, this model incorporates the Distilling operation to compress redundant temporal information, making the model more efficient. PatchTST(Nie et al. 2023) takes a different approach. It draws on the idea of image block division in computer vision, dividing the time series into non-overlapping patches. Then, it uses the Transformer encoder to model these patches, thereby capturing the local features of the time series patterns.

Among MLP-based models, TimeMixer(Wang et al. 2024) has attracted considerable attention due to its concise architecture and competitive performance. By sampling the input sequence at multiple temporal scales and decomposing it into seasonal and trend components, TimeMixer effectively captures multi-scale temporal characteristics. Building upon this work, AMD(Hu et al. 2025) further enhances the modeling capacity by incorporating high-order interactions among multi-scale features. Additionally, it introduces a patch-based partitioning mechanism, enabling MLP-based modeling at the patch level and thereby improving the model's ability to capture intricate local temporal dependencies in univariate time series.

Overall, while MLP-based models are lightweight and computationally efficient, and demonstrate strong capabilities in capturing temporal dependencies, they are generally less effective than Transformer-based models in modeling complex inter-channel dependencies.

2.2 Fast Fourier Transform for TSF

In response to the non-stationarity of time series, the application of FFT in time series forecasting has attracted increasing attention in recent years.

FEDformer (Zhou et al. 2022) introduces a Frequency Enhanced Block, which performs frequency-domain transformations using FFT to model the periodic patterns embedded in the input sequence. In addition, FEDformer proposes a

Frequency Selection Mechanism that selectively models selected frequency components instead of all frequency bands. This sparse frequency modeling strategy effectively reduces computational complexity and enhances robustness against noise.

TimeStacker(Liu et al. 2025b) first divides the time series into nonoverlapping patches, introducing FFT within the module to map the frequency-domain features of each patch, and combines the self-attention mechanism to model the temporal dependencies in the univariate time series. Ultimately, TimeStacker effectively addresses the non-stationarity of time series and enhances the model's capability to capture both local and global periodic patterns through a hierarchical learning framework.

2.3 Mixture of Experts for TSF

MoE is a modular neural network architecture that dynamically selects a subset of specialized sub-models, or experts, to process each input. Typically, a gating network learns to route the input to the most relevant experts, allowing the model to allocate computation adaptively. In sparse MoE, this design enables MoE to scale to very large capacities while keeping computational costs manageable, as only a few experts are activated per input. MoE promotes both specialization and computational efficiency. In the field of TSF, MoE is often used to handle distribution drift

DUET(Qiu et al. 2025) employs a sparse MoE architecture, where each expert is an MLP designed to capture distinct patterns within the time series. By dynamically activating and aggregating a subset of these experts, DUET effectively models temporal dependencies while maintaining computational efficiency.

AMD(Hu et al. 2025) assumes that all temporal patterns captured by the MoE contribute to prediction performance, and therefore adopts a dense MoE architecture, where all experts are activated and their outputs are jointly utilized. Although this helps to capture more comprehensive features and patterns and provides greater robustness, it significantly increases the computational load.

To address these issues in dense MoE, we introduce a shallow shared layer at the bottom of the MoE. The shared layer extracts common low-level features across all experts, significantly reducing the overall parameter count and computational cost. By offloading general feature extraction to the shared layer, each expert can focus on modeling more specialized, high-level patterns, thereby enhancing expert diversity and specialization. This design strikes a balance between computational efficiency and model expressiveness.

3 Proposed Method

3.1 Down-Sampling Block

The changes in time series are often the result of the superposition of long-term trends and short-term disturbances. Downsampling the raw time series to extract information at different scales is mainly to better capture the multi-scale change patterns and characteristics. Among them, high-frequency information contains the detailed patterns of rapid changes, while low-frequency information reflects the slow-changing trend of the sequence.

The original input sequence X already contains fine-grained information, while the coarse-grained information is obtained through average pooling. The i-th sequence can be expressed as:

$$X_0 = \begin{cases} X, & i = 0 \\ \text{AvgPooling}\left(X_i\right), & i = 1, ..., h \end{cases} \tag{1}$$

where $X_i \in \mathbb{R}^{C \times \left[\frac{L}{d_i - 1}\right]}$, h indicates the number of downsampling operations, and d_i denotes the downsampling rate.

3.2 Frequency-based Sparse Attention Block

Fast Fourier Transform. By applying the Fast Fourier Transform (FFT) to convert time-domain data into the frequency domain, we aim to capture global dependencies and periodic patterns more effectively, while also reducing computational complexity. Consider a list containing multi-scale information from the Down-Sampling Block $X_{list} = [X_0, X_1, ..., X_h]$. For any $i \in \{0, 1, \ldots, h\}$, we perform the following transformation:

$$\tilde{X}_i = \text{FFT}\left(X_i\right) \tag{2}$$

$$\mathrm{Q}_i = W_{i,q} \odot \tilde{X}_i, \mathrm{K}_i = W_{i,k} \odot \tilde{X}_i, \mathrm{V}_i = W_{i,v} \times X_i \tag{3}$$

where $W_{i,q}, W_{i,k} \in \mathbb{R}^{[\frac{l_i}{2}]+1}$, $W_v \in \mathbb{R}^{l_i \times l_i^{'}}$, and $\odot$ denotes the Hadamard product. We opt to apply the Hadamard product to the frequency-domain representations of Q and K primarily because traditional linear mappings tend to disrupt the spectral structure of time series. In contrast, the Hadamard operation allows W_q and W_k to function as learnable filters (Liu et al. 2025b), enabling adaptive modulation of the amplitudes of individual frequency components. This helps mitigate the effects of phase variations and better preserve the integrity of the spectral information (Wu 2024).

Sparse Attention Mechanism. To further enhance the model's expressive power and reduce computational complexity, we propose a Top-K filtering strategy integrated with a gating mechanism after computing the full attention scores. This design is motivated by the observation that, in time series data, only a few key dependencies significantly influence the current prediction. Retaining all attention links may introduce noise and increase the risk of overfitting. Therefore, we adopt a sparse attention mechanism in which the gating units identify and preserve the Top-K most informative attention weights, thereby enhancing the model's focus on critical temporal patterns. The specific implementation is as follows:

$$\text{SparseAttnScore} = \text{Gating}\left(\text{FullAttnScore}\right) = \text{Gating}\left(\mathrm{Q}_i \cdot \mathrm{K}_i^T\right) \tag{4}$$

$$\text{Gating}(x) = \text{softmax}\left(\frac{\text{Top-K}_1\left(\text{ReLU}(x), k\right)}{\sqrt{d_x}}\right) \tag{5}$$

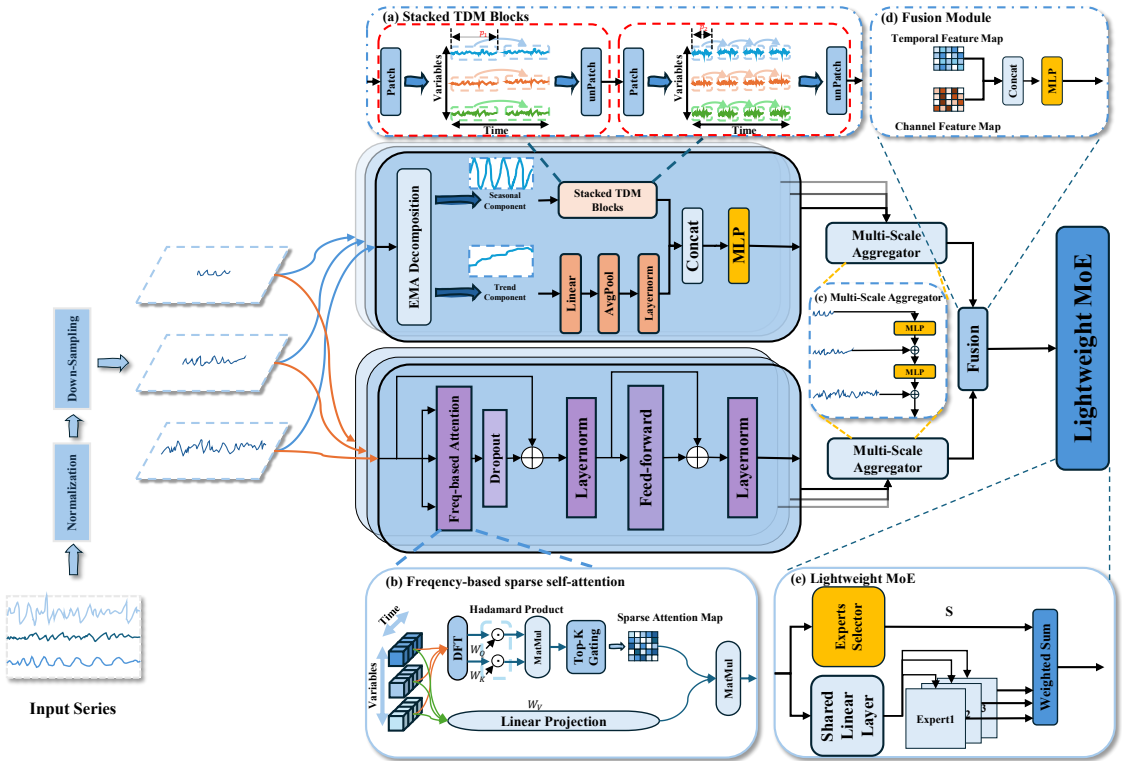


Figure 1: Overall structure of the proposed DDMR. The model adopts a dual-stream architecture. The Transformer-based stream model inter-channel dependency, while the MLP-based stream focuses on modeling temporal dependency. The MA blocks are designed to aggregate information across multi-scale sequences. LMoE block learns multiple predictive functions.

$$\text{Top-K}_1(\text{u}, k) = \begin{cases} \text{-inf}, & \text{if } u < v_k \\ u, & \text{if } u \geq v_k \end{cases} \tag{6}$$

where, v_k represents the k-th largest value. First, calculate the traditional attention score.

3.3 Stacked Temporal Dependency Modeling Blocks

In TSF, the patch operation refers to segmenting the input sequence into non-overlapping or overlapping fragments. However, most existing approaches rely on a single, fixed patch size, which constrains the model's capacity to capture multi-scale temporal dependencies. To overcome this limitation, we propose the Stacked Temporal Dependency Modeling (STDM) block, which partitions the raw sequence into non-overlapping segments using multiple patch sizes. The architecture begins with the largest patch size to prioritize the extraction of global dependencies, and progressively decreases the patch size to enhance the modeling of local temporal dynamics. This hierarchical structure enables a coarse-to-fine feature extraction process—transitioning from global perception to local refinement—thereby improving the model's expressiveness.

Temporal Dependency Modeling Block. Within each Temporal Dependency Modeling (TDM) block, we first apply the patching operation to the input, followed by temporal dependency modeling. Specifically, given an input sequence of TDM $X \in \mathbb{R}^{C \times L}$, and p denotes the patch size. We define the patch operation as:

$$[x_1, x_2, ..., x_{[\frac{L}{p}]}] = Patch_p\left(X\right) \tag{7}$$

where $\frac{L}{p}$ represents the number of patches, $x_i \in \mathbb{R}^{C \times p}$, and x_i do not overlap with each other. Then, we model the time dependency as follows:

$$s_i = f\left(x_i\right) = \begin{cases} x_1, i = 1 \\ x_i + \text{MLP}\left(s_{i-1}\right), i = 2, ..., [\frac{L}{p}] \end{cases} \tag{8}$$

We can obtain the output of a TDM block as follows:

$$X^{'} = \text{Concat}(s_1, \ldots, s_{\lfloor \frac{L}{p} \rfloor}) \tag{9}$$

where $X^{'} \in \mathbb{R}^{C \times L}$. We summarized that:

$$X^{'} = \text{TDM}_p\left(X\right) = \text{F}\left(\text{Patch}_p\left(X\right)\right) \tag{10}$$

where $\text{F}\left(\cdot\right)$ denotes the combination of Eq. (8) and Eq. (9).

Condider a series of p of different size: $[p_1, p_2, ..., p_n]$, where $p_1 > p_2 > ... > p_n$. Then the final output of the STDM can be expressed as:

$$U = TDM_{p_n}\left(...TDM_{p_1}\left(X\right)...\right) \tag{11}$$

Exponential Moving Average. Time series typically exhibit multiple variation patterns, including long-term trends, seasonal fluctuations, and irregular random disturbances. In existing studies (Wu et al. 2021; Zhou et al. 2022; Wang et al. 2024), trend components are commonly extracted using the Simple Moving Average (SMA). To preserve the original sequence length during SMA computation, padding is required. However, this padding operation inevitably introduces artificial values, especially at the boundaries of the sequence, which can significantly distort trend estimation and increase fitting errors. To address this issue, inspired by the design of xPatch (Stitsyuk and Choi 2025), this paper adopts the Exponential Moving Average (EMA) (Gardner Jr 1985) to capture the trend characteristics of time series. The EMA is computed as follows:

$$\begin{cases} s_0 = x_0 \\ s_t = \alpha x_t + (1-\alpha)\, s_{t-1} \end{cases} \tag{12}$$

In summary, when modeling the temporal dependency relationship of a time series, we consider modeling seasonal components and trend components separately. For seasonal components, we use STDM blocks to capture their global dependencies while enhancing local dependencies; For trend component, we employ a specially designed MLP to enhance the trend information. The process of temporal dependency modeling can be summarized as:

$$\begin{aligned} &X_T = EMA\left(X\right), X_S = X - X_T \\ &X_S^{'} = \text{STDM}\left(X_S\right) \\ &X_T^{'} = \text{MLP}\left(X_T\right) = \text{Layernorm}\left(\text{Avgpool}\left(\text{Linear}\left(X_T\right)\right)\right) \\ &\tilde{X} = \text{MLP}\left(\text{Concat}\left(X_S^{'}, X_T^{'}\right)\right) \end{aligned} \tag{13}$$

3.4 Lightweight MoE with Shared Experts

In the time series prediction task, MoE can effectively address the modeling challenges brought about by the distribution drift of time series(Qiu et al. 2025). AMD(Hu et al. 2025) has applied MoE to the field of time series prediction, and its core idea is that different experts can capture different distribution patterns in the time series, and these diverse patterns are valuable for improving prediction accuracy. Therefore, AMD adopts a dense MoE structure, where each expert is activated and participates in the prediction during each training and inference process. However, this approach also has certain limitations. On the one hand, since all experts participate in the forward computation and weight updates during each training step, the computational cost of dense MoE is significantly higher than that of sparse MoE. On the other hand, although increasing the number of experts may enhance the model's ability to capture complex and diverse data distributions and improves prediction accuracy, it also leads to a substantial growth in model parameters, severely impacting training and inference efficiency. Therefore, a key challenge in further optimizing MoE-based time series forecasting models lies in enhancing parameter utilization and reducing computational overhead while maintaining expert diversity.

To build an efficient and scalable MoE architecture, we propose the Lightweight MoE (LMoE) block, which incorporates parameter-sharing expert layers and input-aware nonlinear adjustment factors. This design aims to enhance dynamic modeling capabilities while significantly reducing parameter count and computational cost. In conventional MoE structures, experts operate independently, which improves expressiveness but often leads to parameter redundancy and unstable training. In contrast, LMoE shares a shallow feedforward subnetwork extracting common low-level features across all experts, while each expert's independent network is dedicated to capturing specialized and high-level features.

Before the experts in the expert network are engaged in computation, the model first assigns weights to each expert through a Gating Unit. Our weighting strategy is based on a key assumption: every expert contributes to the prediction task to some extent. Therefore, we adopt dense MoE. Compared to the sparse MoE architecture with selective activation, the dense design not only better leverages the representation capacity of all experts, but also effectively mitigates common issues such as load imbalance and embedding omission often observed in sparse MoE models. The proposed design of our gating mechanism is as follows(Hu et al. 2025):

$$\text{Top-K}_2(u,k) = \begin{cases} \alpha \cdot \ln(u+1), & \text{if } u < v_k \\ \alpha \cdot (\exp(u) - 1), & \end{cases} \tag{14}$$

where v_k denotes the k-th largest value in $\mathbf{u}$. However, the α in AMD is a fixed constant. While it can amplify the weight differences among experts, it lacks the flexibility to adaptively adjust the scaling factor based on the input. To address this limitation, we design a function that dynamically adjusts α according to the characteristics of each input sample, as follows:

$$\alpha = \alpha_{base} \cdot \frac{\text{mean}\left(\text{std}\left(u\right)\right)}{\text{std}\left(u\right) + \epsilon} \tag{15}$$

for batches with large variations, we aim to use a smaller α to suppress excessive disparities among experts. Conversely, for batches with relatively minor variations, a larger α is preferred to enhance the diversity of among experts.

The input sequence of the LMoE block is $U \in \mathbb{R}^{C \times L}$, then the formula for calculating the weights can be expressed as:

$$\mathbf{S} = \text{Softmax}\left(\text{Top-K}_2\left(u,k\right)\right) \tag{16}$$

$$u = \text{Softmax}\left(\text{Linear}\left(U\right)\right) \tag{17}$$

During the prediction process, a shallow neural network that is shared by all experts is first used to uniformly process the input sequence. The handling method is as follows:

$$U^{'} = \text{Linear}\left(U\right) \tag{18}$$

Finally, $U^{'}$ is input into the expert network for parallel processing. By weighting and summing up the results of each expert, the final output can be obtained. The process can be expressed as:

$$\text{Output} = \sum_{i=0}^{m} \mathbf{S}_i \cdot \mathbf{U}_i^{'} \tag{19}$$

where m represents the number of experts.

3.5 Loss Function

The loss function of DDMR consists of three components. (1) For the final forecasting results, Mean Squared Error (MSE) loss ($\mathcal{L}_{\text{pred}} = \sum_{i=0}^{T} \|y_i - \bar{y}_i\|_2^2$)) is used to measure the variance between predicted values and ground truth. (2) For the Top-K gating network in the Transformer and Lightweight MoE block, we apply the cofficient of variation loss function (the former is denoted by $\mathcal{L}_1 = \frac{Var(SA)}{Mean(SA^2)+\epsilon}$, and the latter by $\mathcal{L}_2 = \frac{Var(S)}{Mean(S^2)+\epsilon}$, where ϵ is a small positive constant to prevent numerical instability, and SA denotes $SparseAttnScore$), which enhances the gating mechanism by encouraging both a balanced distribution of attention scores and an even assignment of experts, thereby enhancing the overall performance of DDMR. The total loss function is defined as:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{pred} + \lambda_2 \mathcal{L}_1 + \lambda_3 \mathcal{L}_2 \tag{20}$$

where $\lambda_{1,2,3}$ are hyper-parameters.

4 Experiments

Datasets. We conduct extensive experiments on thirteen real-world multivariate time series datasets to validate the effectiveness. These include Electricity Transformer Temperature datasets (ETTh1, ETTh2, ETTm1, ETTm2) (Zhou et al. 2021b), Weather, Electricity, Exchange-rate (Wu et al. 2022), and Solar-energy (Lai et al. 2018) for long-term forecasting, as well as the PEMS datasets (PEMS03, PEMS04, PEMS07, PEMS08) for short-term forecasting.

Evaluation Metrics. Based on the existing work, we selected the Mean Squared Error (MSE) and Mean Absolute Error (MAE) as the evaluation metrics.

Implementation Details. All experiments were conducted using PyTorch(Paszke et al. 2019) on an NVIDIA RTX 5090 GPU with 32 GB of memory.

Baselines. To ensure fair comparisons in terms of input length and to evaluate each model's achievable performance fairly, we select the latest state-of-the-art and publicly available long-term time series forecasting models with open-source implementations, including Autoformer(Wu et al. 2022), FEDformer(Zhou et al. 2022), SCINet(Liu et al. 2022), DLinear(Liu et al. 2022), TimesNet(Wu et al. 2023), TiDE(Das et al. 2024), Crossformer(Wang et al. 2021), PatchTST(Nie et al. 2023), iTransformer(Liu et al. 2024), TimeMixer(Wang et al. 2024), AMD(Hu et al. 2025), and TimeBase(Huang et al. 2024).

Unified Experimental Settings. To ensure a fair comparison, we employed uniform settings: a lookback length $L = 96$, prediction lengths $T = 24, 36, 48, 60$ for the ILI dataset, and $L = 96$, $T = 96, 192, 336, 720$ for all other datasets.

Results. Comprehensive forecasting results are presented in Tab. 1 and Tab. 2, detailing the model's performance on both long-term and short-term prediction tasks. The best results are highlighted in **bold**, while the second-best are underlined. Lower MSE and MAE values indicate better forecasting accuracy. Our proposed DDMR model achieves the best performance in 52 out of 80 cases and ranks second in 9 additional cases, demonstrating its overall superiority. In long-term forecasting, DDMR exhibits particularly strong results on low-dimensional datasets—such as ETTh1, ETTh2, ETTm1, ETTm2, and ILI—where it nearly achieves state-of-the-art (SOTA) performance across all horizons. However, its effectiveness diminishes on high-dimensional datasets like ECL and Solar, where performance is comparatively less competitive. Additionally, when the lookback window is set to 96, TimeBase underperforms significantly compared to its results reported in the original paper (which used a window of 720). This degradation may stem from TimeBase's relatively small parameter capacity, making it more sensitive to noise and less stable when processing shorter input sequences.

Model Interpretablilty. Fig. 2 presents the original signals of seven variables from the ETTh2 dataset over a selected time period, alongside the sparse attention map generated by DDMR and the corresponding frequency domain representations. The sparse attention mechanism in DDMR, equipped with a gating unit, is designed to focus on the Top-K most relevant variables, thereby improving computational efficiency. As shown, the model effectively captures and highlights critical variable interactions, validating the efficacy of the proposed attention strategy. Fig. 3 illustrates the operational mechanism of the LMoE block. This module dynamically assigns weights to each expert based on the characteristics of the input sequence, enabling more precise predictions. For sequences with similar statistical properties, the assigned weights tend to converge.

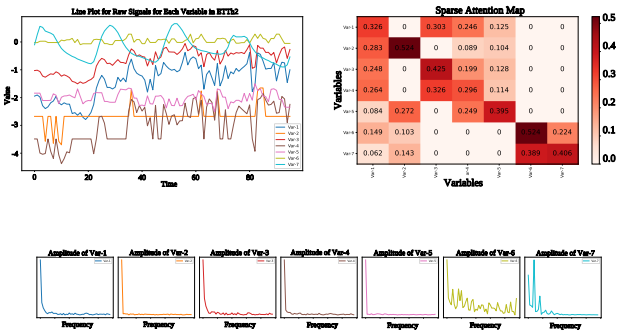


Figure 2: Sparse attention score from ETTh1 dataset calculated based on frequency domain features. Var1-Var7 denote distinct varibales. The bottom graph represents the result of the FFT.

Models		DDMR (Ours)		AMD		TimeBase		Timemixer		iTransformer		PatchTST		Crossformer		TiDE		TimesNet		DLinear		SCINet		FEDformer		Autoformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Exchange	96	**0.084**	**0.202**	0.090	0.206	0.091	0.211	0.090	0.235	0.086	0.206	0.088	0.205	0.256	0.367	0.094	0.218	0.107	0.234	0.088	0.218	0.267	0.396	0.148	0.278	0.197	0.323
	192	0.187	0.304	0.190	0.306	0.197	0.316	0.187	0.343	0.177	**0.299**	**0.176**	**0.299**	0.470	0.509	0.184	0.307	0.226	0.344	**0.176**	0.315	0.351	0.459	0.271	0.315	0.300	0.369
	336	0.352	0.429	0.348	0.427	0.349	0.426	0.353	0.473	0.331	0.417	**0.301**	**0.397**	1.268	0.883	0.349	0.431	0.367	0.448	0.313	0.427	1.324	0.853	0.460	0.427	0.509	0.524
	720	**0.696**	**0.600**	0.710	0.603	1.023	0.764	0.934	0.761	0.847	0.691	0.901	0.714	1.767	1.068	0.852	0.698	0.964	0.746	0.839	0.695	1.058	0.797	1.195	0.695	1.447	0.941
ETTh1	96	**0.369**	**0.385**	0.380	0.394	0.397	0.391	0.375	0.400	0.386	0.405	0.460	0.447	0.423	0.448	0.479	0.464	0.384	0.402	0.397	0.412	0.654	0.599	0.395	0.424	0.449	0.459
	192	**0.429**	**0.414**	0.434	0.422	0.456	0.423	**0.429**	0.421	0.441	0.512	0.477	0.429	0.471	0.474	0.525	0.492	0.436	0.429	0.446	0.441	0.719	0.631	0.469	0.470	0.500	0.482
	336	**0.450**	**0.421**	0.472	0.438	0.502	0.444	0.484	0.458	0.487	0.458	0.546	0.496	0.570	0.546	0.565	0.515	0.491	0.469	0.489	0.467	0.778	0.659	0.530	0.499	0.521	0.496
	720	0.474	**0.454**	**0.471**	0.458	0.503	0.462	0.498	0.482	0.503	0.491	0.544	0.517	0.653	0.621	0.594	0.558	0.521	0.500	0.513	0.510	0.836	0.699	0.598	0.544	0.514	0.512
ETTh2	96	**0.276**	**0.326**	0.287	0.338	0.337	0.376	0.289	0.341	0.297	0.349	0.308	0.355	0.745	0.584	0.400	0.440	0.340	0.374	0.340	0.394	0.707	0.621	0.358	0.397	0.346	0.388
	192	**0.347**	**0.372**	0.387	0.402	0.401	0.405	0.372	0.392	0.380	0.400	0.393	0.405	0.877	0.656	0.528	0.509	0.402	0.414	0.482	0.479	0.860	0.689	0.429	0.439	0.456	0.452
	336	**0.340**	**0.374**	0.406	0.396	0.437	0.441	0.386	0.414	0.428	0.432	0.427	0.436	1.043	0.731	0.643	0.571	0.452	0.452	0.591	0.541	1.000	0.744	0.496	0.487	0.482	0.486
	720	**0.401**	**0.423**	0.409	0.432	0.459	0.477	0.412	0.434	0.427	0.445	0.436	0.450	1.104	0.763	0.874	0.679	0.462	0.468	0.839	0.661	1.249	0.838	0.463	0.474	0.515	0.511
ETTm1	96	**0.313**	**0.338**	0.324	0.361	0.373	0.387	0.320	0.357	0.334	0.368	0.352	0.374	0.404	0.426	0.364	0.387	0.338	0.375	0.346	0.374	0.418	0.438	0.379	0.419	0.505	0.475
	192	0.368	**0.367**	0.368	0.381	0.410	0.408	**0.361**	0.381	0.390	0.393	0.374	0.387	0.450	0.451	0.398	0.404	0.374	0.387	0.382	0.391	0.439	0.450	0.426	0.441	0.553	0.496
	336	**0.389**	**0.388**	0.399	0.402	0.435	0.420	0.390	0.404	0.426	0.420	0.421	0.414	0.532	0.515	0.428	0.425	0.410	0.411	0.415	0.415	0.490	0.485	0.445	0.459	0.621	0.537
	720	0.461	**0.427**	0.462	0.437	0.503	0.461	**0.454**	0.441	0.491	0.459	0.462	0.449	0.666	0.589	0.487	0.461	0.478	0.450	0.473	0.451	0.595	0.550	0.543	0.490	0.671	0.561
ETTm2	96	**0.172**	**0.250**	0.181	0.265	0.188	0.272	0.175	0.258	0.180	0.264	0.183	0.270	0.287	0.366	0.207	0.305	0.187	0.267	0.193	0.293	0.286	0.377	0.203	0.287	0.255	0.339
	192	**0.235**	**0.290**	0.245	0.304	0.251	0.310	0.237	0.299	0.250	0.309	0.255	0.314	0.414	0.492	0.290	0.364	0.249	0.309	0.284	0.361	0.399	0.445	0.269	0.328	0.281	0.340
	336	**0.297**	**0.332**	0.303	0.341	0.310	0.345	0.298	0.340	0.311	0.348	0.309	0.347	0.597	0.542	0.377	0.422	0.321	0.351	0.382	0.429	0.637	0.591	0.325	0.366	0.339	0.372
	720	**0.390**	**0.388**	0.398	0.395	0.412	0.402	0.391	0.396	0.412	0.407	0.412	0.404	1.730	1.042	0.558	0.524	0.408	0.403	0.558	0.525	0.960	0.735	0.421	0.415	0.433	0.432
ECL	96	0.198	0.283	0.170	0.259	0.212	0.278	0.153	0.247	**0.148**	**0.240**	0.190	0.296	0.219	0.314	0.237	0.329	0.168	0.272	0.210	0.302	0.247	0.345	0.193	0.308	0.201	0.317
	192	0.197	0.290	0.175	0.265	0.209	0.280	0.166	0.256	**0.162**	**0.253**	0.199	0.304	0.231	0.322	0.236	0.330	0.184	0.322	0.210	0.305	0.257	0.355	0.201	0.315	0.222	0.334
	336	0.217	0.305	0.193	0.283	0.223	0.295	0.185	0.277	**0.178**	**0.269**	0.217	0.319	0.246	0.337	0.249	0.344	0.198	0.300	0.223	0.319	0.269	0.369	0.214	0.329	0.231	0.443
	720	0.272	0.347	0.235	0.316	0.264	0.327	0.225	**0.310**	0.225	0.317	0.258	0.352	0.280	0.363	0.284	0.373	**0.220**	0.320	0.258	0.350	0.299	0.390	0.246	0.355	0.254	0.361
Weather	96	**0.163**	**0.198**	0.182	0.227	0.169	0.215	**0.163**	0.209	0.174	0.214	0.186	0.227	0.195	0.271	0.202	0.261	0.172	0.220	0.195	0.252	0.221	0.306	0.217	0.296	0.266	0.336
	192	**0.208**	**0.239**	0.228	0.265	0.216	0.257	**0.208**	0.250	0.221	0.254	0.234	0.265	0.209	0.277	0.242	0.298	0.219	0.261	0.237	0.295	0.261	0.340	0.276	0.336	0.307	0.367
	336	0.262	**0.286**	0.281	0.302	0.272	0.300	**0.251**	0.287	0.278	0.296	0.284	0.301	0.273	0.332	0.287	0.335	0.280	0.306	0.282	0.331	0.309	0.378	0.339	0.380	0.359	0.395
	720	0.345	**0.336**	0.350	0.347	0.352	0.350	**0.339**	0.341	0.358	0.347	0.356	0.349	0.379	0.401	0.351	0.386	0.365	0.359	0.345	0.382	0.377	0.427	0.403	0.428	0.419	0.428
Solar	96	0.254	0.259	0.276	0.285	0.618	0.492	**0.189**	0.259	0.203	**0.237**	0.265	0.323	0.232	0.302	0.312	0.399	0.373	0.358	0.290	0.378	0.237	0.344	0.286	0.341	0.456	0.446
	192	0.288	0.274	0.311	0.301	0.642	0.506	**0.222**	0.283	0.233	**0.261**	0.288	0.332	0.371	0.410	0.339	0.416	0.397	0.376	0.320	0.398	0.280	0.380	0.291	0.337	0.588	0.561
	336	0.314	0.286	0.350	0.321	0.637	0.496	**0.231**	0.292	**0.248**	**0.273**	0.301	0.339	0.495	0.515	0.368	0.430	0.420	0.380	0.353	0.415	0.304	0.389	0.354	0.416	0.595	0.588
	720	0.333	0.292	0.353	0.320	0.589	0.471	**0.223**	0.285	0.249	**0.275**	0.295	0.336	0.526	0.542	0.370	0.425	0.420	0.381	0.357	0.413	0.308	0.388	0.380	0.437	0.733	0.633
ILI	24	**1.395**	**0.737**	2.215	1.001	3.648	1.385	1.884	0.821	2.911	1.142	1.692	0.814	3.218	1.159	3.125	1.169	2.315	0.932	2.356	1.021	3.458	1.356	3.237	1.262	3.025	1.235
	36	**1.331**	**0.728**	2.174	0.966	3.696	1.392	1.398	0.743	2.918	1.155	1.414	0.763	3.236	1.162	2.986	0.978	1.977	0.924	2.578	1.056	3.659	1.424	2.598	1.054	2.879	1.069
	48	**1.381**	**0.745**	2.216	0.992	4.291	1.487	1.629	0.802	2.846	1.142	1.749	0.816	3.825	1.274	3.654	1.205	2.235	0.936	2.458	0.987	4.256	1.426	2.582	1.050	2.583	1.024
	60	**1.579**	**0.826**	2.378	1.049	4.800	1.554	1.875	0.875	2.994	1.175	1.642	0.856	5.019	1.583	4.012	1.345	2.015	0.924	2.906	1.189	4.379	1.453	2.827	1.137	2.695	1.103
1st count		**21**	**26**	1	0	0	0	11	1	4	8	2	2	0	0	0	0	1	0	1	0	0	0	0	0	0	0

Table 1: Long-term forecasting task. Full long-term forecasting results with unified lookback window $L = 36$ for ILI dataset, and $L = 96$ for all other datasets.

bu

Models	DDMR		AMD		PatchTST		Crossformer		FEDformer		TimesNet		TiDE		DLinear		MTS-Mixers		RLinear	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PEMS03	**0.080**	**0.185**	0.084	0.198	0.099	0.216	0.090	0.203	0.126	0.251	0.085	0.192	0.178	0.305	0.122	0.243	0.117	0.232	0.126	0.236
PEMS04	0.087	**0.193**	**0.083**	0.198	0.105	0.224	0.098	0.218	0.138	0.262	0.087	0.195	0.219	0.340	0.148	0.272	0.129	0.267	0.138	0.252
PEMS07	0.089	0.195	**0.074**	**0.180**	0.095	0.150	0.094	0.200	0.109	0.225	0.082	0.181	0.173	0.304	0.115	0.242	0.134	0.278	0.118	0.235
PEMS08	**0.083**	**0.185**	0.093	0.206	0.168	0.232	0.165	0.214	0.173	0.273	0.112	0.212	0.227	0.343	0.154	0.276	0.186	0.286	0.133	0.247

Table 2: Short-term forecasting task. The results are obtained using a lookback window $L = 96$ and a prediction horizon $T = 12$.

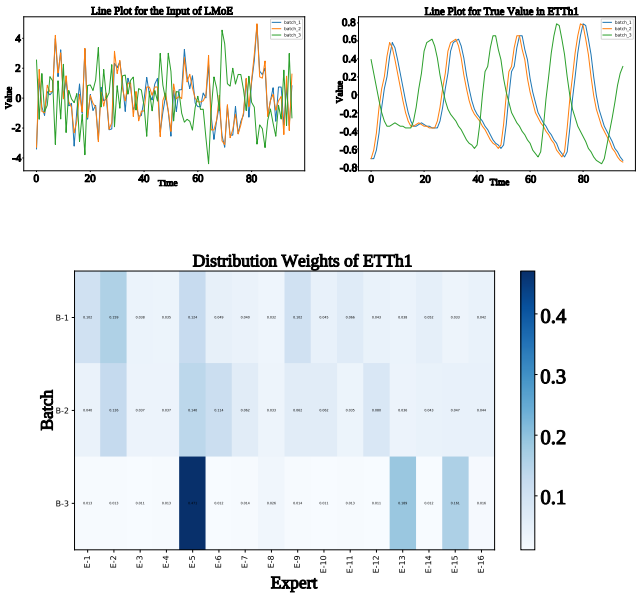


Figure 3: The distribution weights of different batches from ETTh1 in LMoE block. B1-B3 denote distinct batches, while E1-E16 represent different experts.

5 Conclusion

In this paper, we propose the Dual Dependency-Aware Multilscale Representation Learning (DDMR) framework for multivariate time series forecasting. DDMR first decomposes the input time series into multiple temporal scales using average pooling operations, enabling the model to capture both high-frequency fluctuations and low-frequency trends, then captures temporal dependencies and cross-channel relationships using MLP and Transformer modules, respectively. Then, the extracted multi-scale features are adaptively fused using the Multi-Scale Aggregator module, a fully MLP-based component designed to effectively integrate information across different temporal resolutions. Finally, the Lightweight MoE block adaptively and dynamically selects suitable prediction networks to handle distribution shifts. Extensive experiments demonstrate that DDMR achieves superior performance on long-term forecasting tasks across multiple benchmark datasets.

References

Das, A.; Kong, W.; Leach, A.; Mathur, S.; Sen, R.; and Yu, R. 2024. Long-term Forecasting with TiDE: Time-series Dense Encoder. arXiv:2304.08424.

Gardner Jr, E. S. 1985. Exponential smoothing: The state of the art. *Journal of forecasting*, 4(1): 1–28.

Hou, H.; and Yu, F. R. 2024. RWKV-TS: Beyond Traditional Recurrent Neural Network for Time Series Tasks. arXiv:2401.09093.

Hu, Y.; Liu, P.; Zhu, P.; Cheng, D.; and Dai, T. 2025. Adaptive multi-scale decomposition framework for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 17359–17367.

Huang, Q.; Zhou, Z.; Yang, K.; Yi, Z.; Wang, X.; Jiang, W.; and Wang, Y. 2024. TimeBase: The Power of Minimalism in Long-term Time Series Forecasting.

Kong, W.; Dong, Z. Y.; Jia, Y.; Hill, D. J.; Xu, Y.; and Zhang, Y. 2017. Short-term residential load forecasting based on LSTM recurrent neural network. *IEEE transactions on smart grid*, 10(1): 841–851.

Lai, G.; Chang, W.-C.; Yang, Y.; and Liu, H. 2018. Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. arXiv:1703.07015.

Lea, C.; Vidal, R.; Reiter, A.; and Hager, G. D. 2016. Temporal Convolutional Networks: A Unified Approach to Action Segmentation. arXiv:1608.08242.

Liu, M.; Zeng, A.; Chen, M.; Xu, Z.; Lai, Q.; Ma, L.; and Xu, Q. 2022. SCINet: Time Series Modeling and Forecasting with Sample Convolution and Interaction. arXiv:2106.09305.

Liu, P.; Wu, B.; Hu, Y.; Li, N.; Dai, T.; Bao, J.; and tao Xia, S. 2025a. TimeBridge: Non-Stationarity Matters for Long-term Time Series Forecasting. arXiv:2410.04442.

Liu, Q.; Xu, C.; Jiang, W.; Wang, K.; Ma, L.; and Li, H. 2025b. TimeStacker: A Novel Framework with Multilevel Observation for Capturing Nonstationary Patterns in Time Series Forecasting. In *Forty-second International Conference on Machine Learning*.

Liu, Y.; Hu, T.; Zhang, H.; Wu, H.; Wang, S.; Ma, L.; and Long, M. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. arXiv:2310.06625.

Ma, X.; Tao, Z.; Wang, Y.; Yu, H.; and Wang, Y. 2015. Long short-term memory neural network for traffic speed prediction using remote microwave sensor data. *Transportation Research Part C: Emerging Technologies*, 54: 187–197.

Nelson, D. M.; Pereira, A. C.; and De Oliveira, R. A. 2017. Stock market's price movement prediction with LSTM neural networks. In *2017 International joint conference on neural networks (IJCNN)*, 1419–1426. Ieee.

Nie, Y.; Nguyen, N. H.; Sinthong, P.; and Kalagnanam, J. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. arXiv:2211.14730.

Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; Desmaison, A.; Köpf, A.; Yang, E.; DeVito, Z.; Raison, M.; Tejani, A.; Chilamkurthy, S.; Steiner, B.; Fang, L.; Bai, J.; and Chintala, S. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703.

Qiu, X.; Wu, X.; Lin, Y.; Guo, C.; Hu, J.; and Yang, B. 2025. DUET: Dual Clustering Enhanced Multivariate Time Series Forecasting. arXiv:2412.10859.

Rasp, S.; Dueben, P. D.; Scher, S.; Weyn, J. A.; Mouatadid, S.; and Thuerey, N. 2020. WeatherBench: a benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems*, 12(11): e2020MS002203.

Smyl, S.; Dudek, G.; and Pełka, P. 2021. ES-dRNN: A Hybrid Exponential Smoothing and Dilated Recurrent Neural Network Model for Short-Term Load Forecasting. arXiv:2112.02663.

Stitsyuk, A.; and Choi, J. 2025. xPatch: Dual-Stream Time Series Forecasting with Exponential Seasonal-Trend Decomposition. arXiv:2412.17323.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Wang, S.; Wu, H.; Shi, X.; Hu, T.; Luo, H.; Ma, L.; Zhang, J. Y.; and Zhou, J. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616*.

Wang, W.; Yao, L.; Chen, L.; Lin, B.; Cai, D.; He, X.; and Liu, W. 2021. CrossFormer: A Versatile Vision Transformer Hinging on Cross-scale Attention. arXiv:2108.00154.

Wu, H. 2024. Revisiting Attention for Multivariate Time Series Forecasting. arXiv:2407.13806.

Wu, H.; Hu, T.; Liu, Y.; Zhou, H.; Wang, J.; and Long, M. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. arXiv:2210.02186.

Wu, H.; Xu, J.; Wang, J.; and Long, M. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34: 22419–22430.

Wu, H.; Xu, J.; Wang, J.; and Long, M. 2022. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. arXiv:2106.13008.

Wu, Z.; Pan, S.; Long, G.; Jiang, J.; Chang, X.; and Zhang, C. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. arXiv:2005.11650.

Xu, N.; Kosma, C.; and Vazirgiannis, M. 2023. TimeGNN: Temporal Dynamic Graph Learning for Time Series Forecasting. arXiv:2307.14680.

Zeng, A.; Chen, M.; Zhang, L.; and Xu, Q. 2022. Are Transformers Effective for Time Series Forecasting? arXiv:2205.13504.

Zhou, H.; Zhang, S.; Peng, J.; Zhang, S.; Li, J.; Xiong, H.; and Zhang, W. 2021a. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, 11106–11115.

Zhou, H.; Zhang, S.; Peng, J.; Zhang, S.; Li, J.; Xiong, H.; and Zhang, W. 2021b. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. arXiv:2012.07436.

Zhou, T.; Ma, Z.; Wen, Q.; Wang, X.; Sun, L.; and Jin, R. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*, 27268–27286. PMLR.

Appendix

A Datasets

We conduct experiments on nine real-world multivariate time series datasets to evaluate the performance of the proposed DDMR model:

- **Electricity Transformer Temperature (ETT)**: which consists of four subsets: ETTm1 and ETTm2 (15-minute level), and ETTh1 and ETTh2 (hourly level). These datasets record the oil temperature of power transformers along with time-related features.
- **Weather**: collected from Germany, includes 21 variables such as temperature, humidity, wind speed, visibility, pressure, dew point, and various radiation and precipitation measurements.
- **Electricity**: describes the hourly electricity consumption data of 321 clients from 2012 to 2014.
- **Exchange-rate**: collects the panel data of daily exchange rates from 8 countries from 1990 to 2016.
- **Solar-energy**: contains the solar power production records from 137 photovoltaic (PV) plants in 2006.
- **ILI**: records the weekly number of patients and influenza-like illness ratio in the USA between 2002 and 2021.
- **PEMS**: The dataset consists of four traffic network subsets — PeMS 03, PeMS 04, PeMS 07, and PeMS 08 — sourced from the California Performance Measurement System (PeMS), which monitors traffic conditions in four distinct regions of California. Data is recorded every five minutes, producing 12 entries per hour and a total of 288 observations each day.

More detailed dataset descriptions are shown in Tab. 3.

B The Function of Eq. 14

In this subsection, we want to explain the reason to choose Eq. 14 as the key point of gating unit.

To facilitate a clearer understanding of the proposed mechanism, we first analyze the single-variable case. Suppose the input to the Top-K gated unit is denoted as $u \in \mathbb{R}^{1\times n}$, where n represents the total number of experts. The primary objective of this module is to assign larger weights to the most informative experts—those with relatively larger u-values—while assigning smaller weights to the less relevant ones. Additionally, we aim to endow the model with the ability to dynamically regulate the degree of sparsity in expert selection based on the input characteristics.

A standard softmax activation, though widely used for normalization and ensuring differentiability, lacks the flexibility to modulate sparsity levels explicitly. To overcome this limitation, we first apply a softmax function to the input vector u to obtain a normalized attention distribution $u' = [u'_1, u'_2, \ldots, u'_n]$, where each $u'_i \in (0, 1]$ and $\sum_{i=1}^{n} u'_i = 1$. Let v_k denote the k-th largest value in u', and suppose $u'_p \geq v_k > u'_q$ for some indices p and q. It is straightforward to verify that, for any $x \in (0, 1]$, the following strict inequality holds:

$$\exp(x) - 1 > x > \ln(x + 1)$$

and both $\exp(x) - 1$ and $\ln(x + 1)$ are strictly increasing and differentiable on this domain. This inequality forms the theoretical foundation of our transformation strategy.

To explicitly control the separation between selected and unselected experts, we introduce a positive scaling hyperparameter $\alpha > 0$, and design a piecewise transformation function as follows:

$$\tilde{u}_i = \begin{cases} \alpha(\exp(u'_i) - 1), & \text{if } u'_i \text{ is among the top-}k \text{ values} \\ \alpha \ln(u'_i + 1), & \text{otherwise} \end{cases}$$

This formulation amplifies the difference between top-scoring experts and the rest. Specifically, for top-k entries, we apply the exponential-based function $\exp(x) - 1$, which increases rapidly and thus accentuates prominent experts. For the remaining entries, we employ the logarithmic transformation $\ln(x + 1)$, which grows more slowly and suppresses the contribution of minor experts.

The scaling factor α plays a crucial role in tuning the sparsity level:

- A larger α sharpens the distribution, promoting a sparser expert selection by boosting the relative difference between top and non-top scores.
- A smaller α, in contrast, flattens the distribution, encouraging more evenly weighted participation of all experts, leading to a denser MoE configuration.

Finally, we apply the softmax function to the transformed vector $\tilde{u} = [\tilde{u}_1, \tilde{u}_2, \ldots, \tilde{u}_n]$ to obtain the final expert weights:

$$s_i = \frac{exp\,(\tilde{u}_i)}{\sum_{j=1}^{n} exp\,(\tilde{u}_j)}$$

This final step ensures that the output remains a valid probability distribution while incorporating both the learned importance scores and the desired level of sparsity.

In summary, this formulation enables the model to adaptively and dynamically control expert utilization based on both the input distribution and the hyperparameter α, offering a flexible and efficient approach to expert allocation in Mixture-of-Experts architectures.

C The Detailed Algorithmic process of DDMR

The algorithm flow of DDMR is shown in Algorithm 1.

Dataset	Dim	Prediction Length	Dataset Size	Frequency
ETTh1, ETTh2	7	{96,192,336,720}	(8545,2881,2881)	Hourly
ETTm1, ETTm2	7	{96,192,336,720}	(34465,11521,11521)	15 min
Weather	21	{96,192,336,720}	(36792,5271,10540)	10 min
Electricity	321	{96,192,336,720}	(18317,2633,5261)	Hourly
Exchange-rate	8	{96,192,336,720}	(5120,665,1422)	Daily
Solar-Energy	137	{96,192,336,720}	(36601,5161,10417)	10 min
ILI	7	{24,36,48,60}	(617,74,170)	Hourly
PEMS03	358	12	(15617,5135,5135)	5 min
PEMS04	307	12	(10172,3375,3375)	5 min
PEMS07	883	12	(16911,5622,5622)	5 min
PEMS08	170	12	(10690,3548,265)	5 min

Table 3: Detailed dataset descriptions.

D Experiment Details

Metric Details. In order to comprehensively compare the performance of different models on the same dataset, we choose Mean Absolute Error (MAE) and Mean Squared Error (MSE) as evaluation metrics. The calculations of these metrics are:

$$MAE = \frac{1}{T}\sum_{i=0}^{T}|\tilde{y}_i - y_i|$$

$$MSE = \frac{1}{T}\sum_{i=0}^{T}(\tilde{y}_i - y_i)^2$$

Baselines. We briefly describe the selected baselines:

(1) AMD(Hu et al. 2025): is a model entirely based on MLP, incorporating the Multi-Scale Decomposable Mixing (DDM) block to model the high-order interaction relationships of multi-scale sequences.

(2) TimeBase(Huang et al. 2024): is a representative of minimalism, achieving efficient prediction by extracting the core basic temporal components.

(3) Timemixer(Wang et al. 2024): is a model based on MLP, exploring the interaction relationships between multi-scale sequences based on the decomposition concept, in order to improve complementary prediction.

(4) iTransformer(Liu et al. 2024): by modeling in the channel dimension rather than the time dimension, it is possible to more efficiently capture the dependencies between variables and improve the prediction performance.

(5) PatchTST(Nie et al. 2023): is a Transformer-based model, dividing time series into patches. It can significantly improve the prediction performance while reducing the computational complexity.

(6) Crossformer(Wang et al. 2021): by means of the attention mechanism across time slices and the hierarchical modeling structure, long-range temporal dependencies can be effectively captured, thereby improving the accuracy and efficiency of long sequence time series prediction.

(7) TiDE(Das et al. 2024): is a time series prediction model based on MLP with an encoder-decoder structure.

(8) TimesNet(Wu et al. 2023): is a model based on CNN and time-domain decomposition. By modeling the time series with multiple frequencies, it utilizes multiple time blocks to capture multi-scale temporal patterns.

(9) DLinear(Zeng et al. 2022): only use a single-layer linear model to capture the relationship between the input sequence and the output sequence

(10) SCINet(Liu et al. 2022): employs a multi-level stacking approach, dividing the input time series into several subsequences and extracting temporal features layer by layer.

(11) FEDformer(Zhou et al. 2022): divides the input sequence into trend terms and periodic terms, and utilizes fourier transformation to enhance the Transformer's ability to model frequency-domain features.

(12) Autoformer(Wu et al. 2022): is based on the sequence decomposition module of self-attention mechanism, which separately modeling the trend and residual terms in the sequence.

E Ablation Study

To validate the effectiveness of each component in DDMR, we conducted comprehensive ablation studies on the ETTm2 and Weather datasets. The results are presented in Tab. 4.

In DDMR, the Transformer-based stream captures channel dependencies, whereas the MLP-based stream models temporal dependencies. Depending on the characteristics of the prediction task, both channel-independent and cross-channel dependency strategies can potentially achieve state-of-the-art performance. We conduct experiments by removing either the Transformer-based stream (**w/o Transformer-based stream**) or the MLP-based stream (**w/o MLP-based stream**). The results demon-

Algorithm 1: The Detailed Algorithmic Process of DDMR

Input: Historical look-back window $X \in \mathbb{R}^{L \times C}$
Parameter: Optional list of parameters
Output: Your algorithm's output
$X = \text{RevIN}(X, \text{norm})$
$X = X^T$
$X_{\text{list}} = \text{Down-Sampling}(X)$
$\text{Output}_1 = []$
for i, x in enumerate (X_{list}) **do**
  $x_{freq} = \text{DFT}\,(x)$
  $x_{amplitude} = \text{abs}\,(x_{freq})$
  $Q, K = W_Q^i \cdot x_{amplitude}, W_K^i \cdot x_{amplitude}$
  $V = \text{matmul}\left(x_{amplitude}, W_V^i\right)$
  $A = \text{matmul}\left(Q, K^T\right)$
  $A_{sparse} = \text{Top-K}\,(A)$
  $x_{new} = \text{matmul}\,(A_{sparse}, V)$
  $x_{new} = \text{dropout}\,(x_{new}) + x$
  $x_{new} = y = \text{Layernorm}\,(x_{new})$
  $x_{output} = \text{Layernorm}\,(\text{FeedForward}\,(y) + x_{new})$
  $\text{Output}_1.\text{append}\,(x_{output})$
end for
$\text{Output}_2 = []$
for i, x in enumerate (X_{list}) **do**
  $x_T, x_S = EMA\,(x)$
  $x_T^{'} = \text{StackedTDMBlocks}\,(x_S)$
  $x_S^{'} = \text{Layernorm}\,(\text{Avg-Pool}\,(\text{Linear}\,(x_T)))$
  $x_{new} = \text{MLP}(\text{concat}\left(x_S^{'}, x_T^{'}\right))$
  $\text{Output}_2.\text{append}\,(x_{new})$
end for
$x_c = \text{Multi-Scale Aggregator}\,(\text{Output}_1)\,, x_c \in \mathbb{R}^{C \times L}$
$x_t = \text{Multi-Scale Aggregator}\,(\text{Output}_2)\,, x_t \in \mathbb{R}^{C \times L}$
$U = \text{MLP}\,(\text{concat}\,(x_c, x_t))\,, U \in \mathbb{R}^{C \times L}$
$\hat{Y} = \text{Lightweight MoE}\,(U)\,, \hat{Y} \in \mathbb{R}^{L \times C}$
$\hat{Y} = \hat{Y}^T$
$\hat{Y} = \text{RevIn}\left(\hat{Y}, denorm\right)$
return $\hat{Y}$

strate that jointly modeling cross-channel dependency and temporal dependency in multivariate time series can enhance prediction accuracy. However, when we replaced the sparse attention mechanism with **Full Attention Score** without changing the overall model structure, the performance dropped significantly. This suggests that, for certain datasets, placing excessive emphasis on modeling cross-channel dependencies may introduce additional noise rather than improving accuracy.

To demonstrate that stacked TDM blocks facilitate comprehensive modeling of both global and local temporal dependencies in time series, we conducted an experiment using only a **Single TDM** block. When provided with a single fixed patch, the model's performance degraded.

In order to demonstrate the role of LMoE in prediction, we chose to remove LMoE (**w/o LMoE**)and conduct the experiments. The decline in the model's prediction performance suggests that the MoE structure may enhance robustness to distribution shifts, thereby improving overall forecasting accuracy.

F Extra Experiments

Tab. 5 presents the overall results of DDMR on six datasets—ETTh1, ETTh2, ETTm1, ETTm2, Weather, and Exchange—under five different random seeds. The experimental results demonstrate that DDMR achieves stable and consistent performance across all settings.

Fig. 4 presents the results of testing with different numbers of experts on the ETTm1 dataset. The findings indicate that increasing the number of experts does not always lead to better performance. For certain datasets, performance can improve as the number of experts increases within a specific range. However, beyond a certain threshold, further increasing the number of experts yields little to no additional benefit in prediction accuracy

G Visualization

Fig. 5 and 6 illustrate the prediction results of DDMR on the ETTh2 and ETTm2 datasets, respectively. The visualizations demonstrate that DDMR is capable of closely capturing the temporal patterns and accurately forecasting future values. These results provide further evidence of the model's robustness and effectiveness across different types of time series data.

H Discussions on Limitations and Future Improvement

The following are the main limitations of this model and the corresponding directions for improvement:

(1) Frequency-domain enhancement: Although the frequency-domain-based self-attention mechanism proposed in this paper can effectively uncover the inter-channel dependencies in multi-variable time series and enhance the model efficiency through sparse gating, it still has certain limitations. Currently, when the model performs frequency-domain modeling, it only uses the spectral amplitude information as the basis for constructing queries and keys, without explicitly considering the phase information, thereby limiting the comprehensive description of the periodic structure and non-stationary features of the sequence. Moreover, the frequency-domain representation is only related to the amplitude of the input signal and lacks deep interaction with the original time-domain sequence, which may lead to a semantic gap between the frequency-domain attention output and the time-domain context, affecting the feature fusion effect. In future work, we plan to incorporate a complex-valued attention-based frequency-domain modeling approach to enable the collaborative representation of amplitude and phase information(Wu et al. 2021), thereby enhancing the expressiveness of frequency-domain dependency modeling.

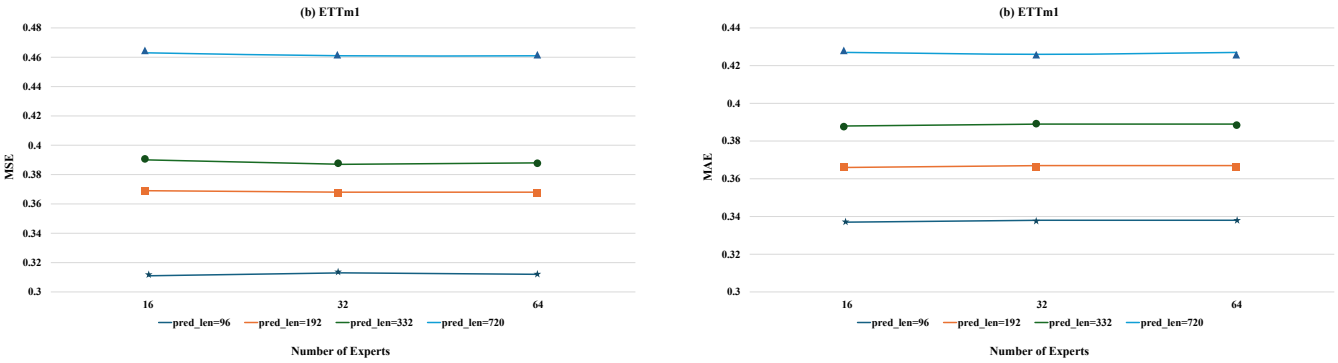
(2) Patching: Although the proposed frequency-domain attention mechanism can effectively model the global dependencies across channels, it still requires a unified processing of the entire input sequence. Even when operating in the frequency domain, compared to the attention calculation in the

Models		DDMR (Ours)		w/o Transformer-based Stream		w/o MLP-based Stream		w/o LMoE		Full Attention Score		Single TDM		Sparse LMoE		w/o $\mathcal{L}_1$		w/o $\mathcal{L}_2$		w/o $\mathcal{L}_1$ & $\mathcal{L}_2$	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm2	96	**0.172**	**0.250**	0.183	0.261	0.176	0.255	0.178	0.257	0.194	0.273	0.180	0.255	0.179	0.260	0.181	0.254	0.174	0.255	0.150	0.206
	192	**0.235**	**0.290**	0.247	0.299	0.240	0.296	0.240	0.298	0.258	0.304	0.236	0.292	0.240	0.295	0.237	0.294	0.239	0.295	0.199	0.248
	336	**0.297**	**0.332**	0.303	0.339	0.302	0.335	0.303	0.336	0.311	0.344	0.303	0.336	0.305	0.338	0.302	0.295	0.306	0.345	0.248	0.287
	720	**0.390**	**0.388**	0.406	0.401	0.395	0.394	0.394	0.395	0.403	0.402	0.396	0.393	0.396	0.392	0.412	0.401	0.398	0.396	0.320	0.339
Weather	96	**0.163**	**0.198**	0.168	0.204	0.178	0.214	0.175	0.210	0.186	0.210	0.168	0.205	0.172	0.207	0.171	0.206	0.175	0.215	0.137	0.234
	192	**0.208**	**0.239**	0.214	0.244	0.221	0.254	0.220	0.252	0.221	0.252	0.214	0.248	0.219	0.248	0.217	0.248	0.227	0.257	0.157	0.251
	336	**0.262**	**0.286**	0.265	0.290	0.266	0.293	0.265	0.290	0.275	0.298	0.263	0.290	0.268	0.294	0.279	0.295	0.273	0.297	0.171	0.267
	720	**0.345**	**0.336**	0.350	0.346	0.351	0.349	0.349	0.343	0.358	0.349	0.354	0.348	0.353	0.343	0.353	0.349	0.356	0.348	0.203	0.298

Table 4: Component ablation of DDMR on ETTm2 and Weather.

Datasets	ETTh1		ETTm1		Weather	
Metric	MSE	MAE	MSE	MAE	MSE	MAE
96	0.3690±0.0001	0.3851±0.0002	0.3131±0.0005	0.3832±0.0004	0.1632±0.0002	0.1981±0.0003
192	0.4293±0.0004	0.4141±0.0004	0.3681±0.0003	0.3673±0.0002	0.2082±0.0004	0.2391±0.0003
336	0.4502±0.0003	0.4211±0.0007	0.3892±0.0002	0.3880±0.0002	0.2621±0.0002	0.2864±0.0004
720	0.4744±0.0006	0.4542±0.0003	0.4612±0.0004	0.4273±0.0001	0.3452±0.0003	0.3360±0.0002
Datasets	ETTh2		ETTm2		Exchange	
Metric	MSE	MAE	MSE	MAE	MSE	MAE
96	0.2762±0.0003	0.3263±0.0005	0.1722±0.0004	0.2503±0.0003	0.0841±0.0003	0.2022±0.0005
192	0.3472±0.0002	0.3721±0.0006	0.2351±0.0006	0.2903±0.0008	0.1873±0.0005	0.3042±0.0003
336	0.3402±0.0004	0.3742±0.0003	0.2971±0.0004	0.3323±0.0004	0.3523±0.0002	0.4293±0.0004
720	0.4013±0.0002	0.4231±0.0003	0.3901±0.0007	0.3882±0.0006	0.6960±0.0004	0.6002±0.0002

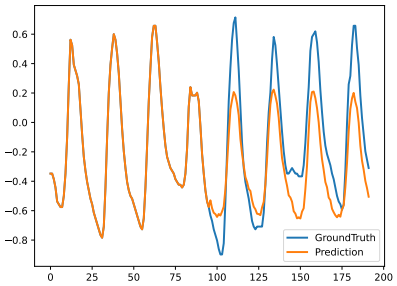
Table 5: Robustness of DDMR performance. The results are obtained from five random seeds.



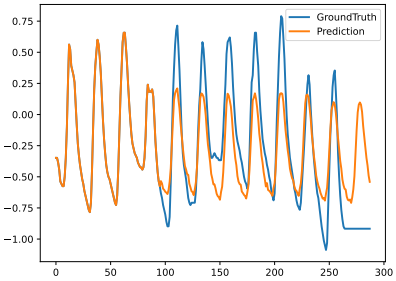
(a) MSE under different number of experts

(b) MAE under different number of experts

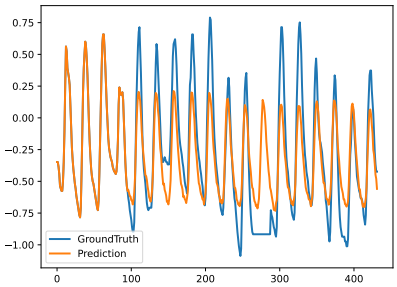
Figure 4: Hyper-Parameter Sensitivity with respect to the number of experts. The results are recorded with the all four predict length.



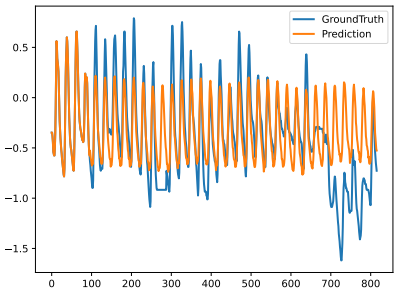
(a) T=96



(b) T=192

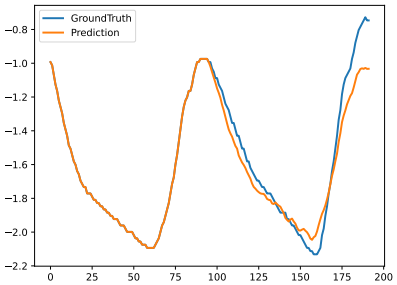


(c) T=336

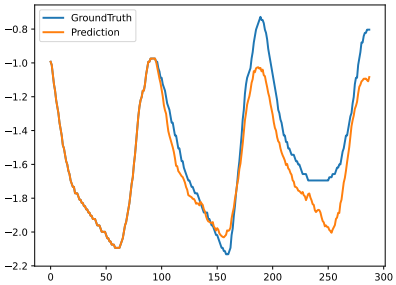


(d) T=720

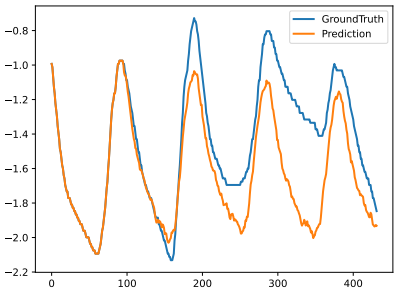
Figure 5: Visualizations of DDMR on ETTh2 dataset under different prediction lengths.



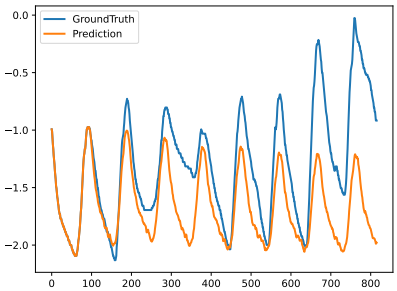
(a) T=96



(b) T=192



(c) T=336



(d) T=720

Figure 6: Visualizations of DDMR on ETTm2 dataset under different prediction lengths.

time domain, it already has certain efficiency advantages. However, for longer input sequences, the computational and memory costs are still significant. Moreover, modeling the entire sequence as a whole may mask the local key dynamics in the time series, such as sudden events, short-term fluctuations, or local trends, thereby weakening the model's ability to perceive non-stationarity or local changes. Therefore, we will focus on the block-based frequency-domain attention mechanism as an improvement direction(Liu et al. 2025b,a). This method divides the long sequence into several non-overlapping blocks, and independently calculates the frequency-domain attention within each block, thereby reducing the complexity of the attention mechanism from $O(L^2)$ to $O(\frac{L^2}{N})$, where L is the sequence length and N is the number of blocks. The local Fourier transform of each block helps the model capture local features in the frequency domain, thereby enhancing its modeling ability for non-stationarity and multi-scale patterns. Additionally, this block-level strategy also enhances the flexibility and adaptability of the model, making it more suitable for handling the local changes and dynamic structures that are widely present in the real world.

Reproducibility Checklist

Example: If a question appears as

```
\question{Proofs of all novel claims
are included} {(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel claims
are included} {(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) no

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) Type your response here

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) Type your response here

2.4. Proofs of all novel claims are included (yes/partial/no) Type your response here

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) Type your response here

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) Type your response here

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) Type your response here

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) Type your response here

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) no

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) yes

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) yes

3.6. All datasets drawn from the existing literature

(potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) no

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) yes

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) yes

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) NA

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) yes

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) no

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) no

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) no